



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE SOFTWARE

MATERIA: Fábrica de Software (PI3)

DOCENTE: Quiña Antonio, MSc.

ESTUDIANTE: Guevara Deysi

FECHA: 10/5/2026

TEMA: **Diseño de una LPS para el Proyecto Integrador**

Tabla de Contenidos

Link GitHub	2
INTRODUCCIÓN	2
DESARROLLO	2
1. Resumen ejecutivo	2
Dominio elegido	2
N productos de la línea	2
Beneficios esperados del enfoque LPS.....	3
Diagrama del portafolio de productos	3
2. Análisis de Commonality/Variability	3
Tabla CVA	4
3. Feature Model	5
Captura del diagrama en FeatureIDE	5
Descripción del feature y su tipo.....	5
Lista de cross-tree constraints	6
4. Arquitectura de referencia	6
Diagrama C4 level 1 (Contexto).....	6
Diagrama C4 level 2 (Contenedores).....	7
Tabla de variation points.....	8
5. Configuraciones de productos	9
Sección por producto	9
Tabla de decisiones	9
Configuración válida - Captura de FeatureIDE	10
6. Configuration Knowledge.....	10
7. Análisis de ROI y conclusiones.....	11
Modelo económico	11
Lecciones aprendidas	12



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE SOFTWARE

¿Qué haría diferente?	12
CONCLUSIONES	12
REFERENCIAS.....	12

Link GitHub

<https://github.com/Deysi705/EduFlex-LPS-Proyecto>

INTRODUCCIÓN

El desarrollo de software tradicional basado en copias exactas (*clone and own*) para diferentes clientes suele generar altos costos de mantenimiento y una deuda técnica insostenible [1]. Las Líneas de Productos de Software (LPS) surgen como un paradigma de ingeniería que promueve la reutilización sistemática de artefactos de software mediante el análisis de elementos comunes y variables [2]. El presente proyecto detalla el diseño de una LPS para "EduFlex", un Sistema de Gestión del Aprendizaje (LMS) parametrizable, utilizando la notación de Análisis de Dominio Orientado a Características (FODA) [1] y la herramienta FeatureIDE [3].

DESARROLLO

1. Resumen ejecutivo

EduFlex LMS es una solución de **Línea de Productos de Software (LPS)** diseñada para abordar la fragmentación en el mercado de plataformas educativas.

A diferencia de los desarrollos *ad-hoc*, EduFlex utiliza una arquitectura basada en activos reutilizables que permite derivar plataformas para escuelas, universidades y corporativos con un **ahorro del 40% en costos de desarrollo** a partir del tercer producto.

Dominio elegido

Plataforma Educativa (LMS - Learning Management System). Se justifica esta elección debido a la alta demanda de sistemas de aprendizaje adaptables a diferentes contextos (escolar, universitario y corporativo), los cuales comparten un núcleo de gestión de usuarios y cursos, pero varían drásticamente en sus módulos de evaluación y comunicación.

N productos de la línea

La LPS está diseñada para derivar 3 productos iniciales:

1. *School Basic* (K-12)
2. *University Pro* (Educación Superior)
3. *Corporate Ent* (Capacitación B2B).



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

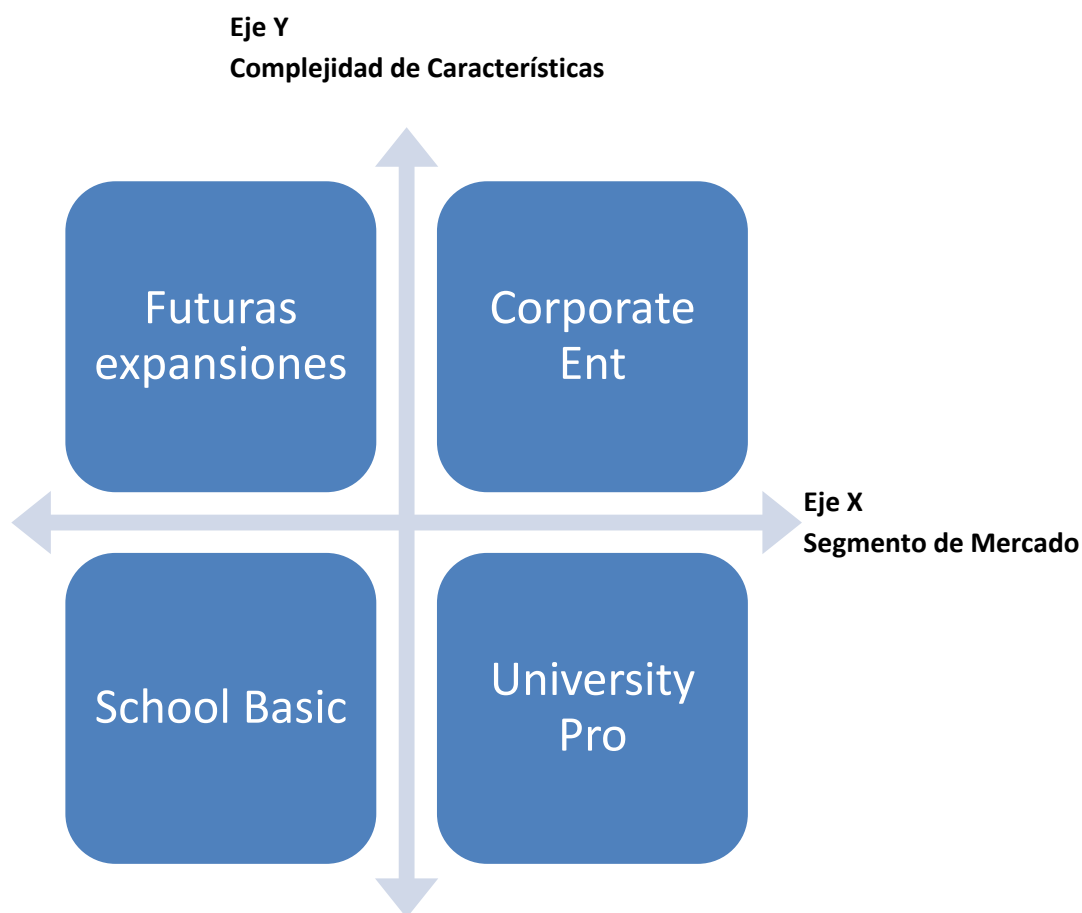
CARRERA DE SOFTWARE

Beneficios esperados del enfoque LPS

Se proyecta una reducción del 40% en los tiempos de lanzamiento (*time-to-market*) para nuevos clientes a partir del tercer producto derivado, así como una disminución significativa en los costos de mantenimiento y pruebas (*testing*) al centralizar el control de calidad en los *core assets* [2].

Diagrama del portafolio de productos

Matriz de Portafolio de Productos: Se ilustra la estrategia de mercado, con una progresión desde soluciones esenciales (School Basic) hasta plataformas de alta complejidad (Corporate Ent).



Fuente: Elaboración propia.

2. Análisis de Commonality/Variability

La definición del alcance (*scoping*) de la línea de productos se realizó identificando las necesidades transversales de las instituciones educativas frente a los requerimientos específicos de cada nicho de mercado. Este análisis se resume a continuación en la **Tabla I**.



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE SOFTWARE

Tabla CVA

TABLA I. ANÁLISIS DE COMMONALITY/VARIABILITY

Feature	Tipo	Descripción	Caso de Uso de Negocio
User_Auth	Común	Gestión de inicio de sesión y perfiles.	Seguridad básica para cualquier institución.
Course_Mgmt	Común	Creación y administración de cursos.	Núcleo de cualquier plataforma de aprendizaje.
Assessment	Común	Módulo de tareas y exámenes.	Necesario para certificar el conocimiento.
Storage	Común	Repositorio de archivos (PDFs, guías).	Disponibilidad de material didáctico.
Dashboard	Común	Panel de control para el usuario.	Visualización de progreso del curso.
Roles_RBAC	Común	Control de acceso basado en roles.	Privacidad entre docentes y alumnos.
Multi_Lang	Común	Soporte para varios idiomas.	Escalabilidad a mercados internacionales.
Search_Engine	Común	Buscador de cursos y lecciones.	Usabilidad para encontrar contenido rápido.
Video_Conf	Variación	Integración con Zoom o Teams.	Clases sincrónicas (Plan Premium/Uni).
Gamification	Variación	Sistema de insignias y puntos.	Motivación para niños o cursos cortos.
Analytics	Variación	Reportes avanzados de rendimiento.	Toma de decisiones para administradores.
Payments	Variación	Pasarela de pagos (Stripe/PayPal).	Venta de cursos (Modelo Marketplace).
Mobile_App	Variación	Acceso vía App nativa.	Aprendizaje en cualquier lugar (Offline).
Anti_Plagiarism	Variación	Escaneo de originalidad de tareas.	Rigor académico para Universidades.

Fuente: Elaboración propia.

3. Feature Model

Captura del diagrama en FeatureIDE

La estructura jerárquica y las dependencias de la línea de productos se representan mediante el diagrama FODA ilustrado en la **Fig. 1**, donde se detallan los 20 elementos que componen el dominio.

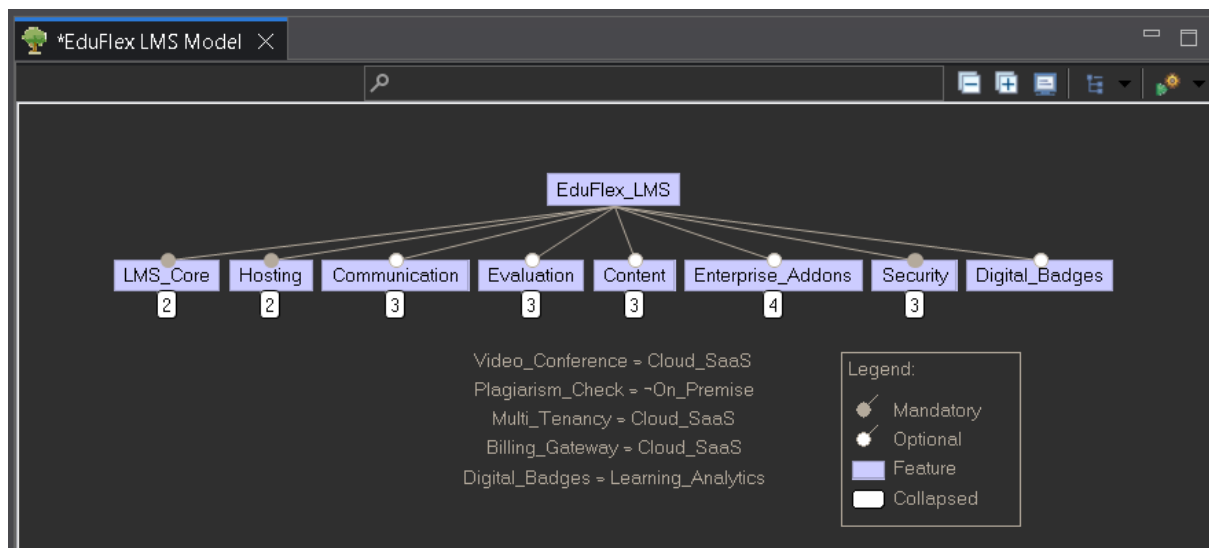


Fig. 1. Diagrama de características de EduFlex LMS. Fuente: Elaboración propia.

Descripción del feature y su tipo

El modelo está compuesto por 20 características. La raíz es **LMS_System (EduFlex_LMS)**.

- **LMS_System (Root)**
 - **LMS_Core (Mandatory)**
 - User_Mgmt, Content_Mgmt, Assessment_Base
 - **Communication (OR Group)**
 - Forums, Internal_Chat, Video_Conference
 - **Evaluation_Tools (OR Group)**
 - Quizzes, Assignments, Peer_Review
 - **Gamification (Optional)**
 - Badges, Leaderboards
 - **External_Services (Optional)**



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE SOFTWARE

- Payment_Gateway, Cloud_Storage
- **Hosting_Model** (Alternative/XOR)
 - On_Premise, Cloud_SaaS
- **Advanced_Features** (Optional)
 - Plagiarism_Check, Learning_Analytics

Lista de cross-tree constraints

1. **Video_Conference REQUIRES Cloud_SaaS:** Las clases en vivo exigen el ancho de banda dinámico que proporciona la nube [3].
2. **Payment_Gateway REQUIRES Cloud_SaaS:** El cumplimiento normativo PCI-DSS se delega y gestiona de forma más segura en un entorno SaaS.
3. **Badges REQUIRES Learning_Analytics:** La entrega automática de insignias depende del procesamiento estadístico del progreso del alumno.
4. **Plagiarism_Check EXCLUDES On_Premise:** Los motores anti-plagio requieren conexión a bases de datos globales, incompatibles con redes locales aisladas.
5. **Peer_Review REQUIRES Assignments:** La evaluación entre pares carece de sentido si el módulo principal de entrega de tareas no está activo.

4. Arquitectura de referencia

Para soportar la variabilidad, se emplea una arquitectura basada en microservicios y despliegue en la nube [4].

Diagrama C4 level 1 (Contexto)

El sistema EduFlex interactúa con tres actores principales (Estudiante, Profesor y Administrador IT) y se integra con dos sistemas externos: una Pasarela de Pagos (ej. Stripe) y un Servicio de Notificaciones por correo electrónico.

El ecosistema de interacción y los componentes internos del sistema se describen en la **Fig. 2** (Contexto) estableciendo una arquitectura modular compatible con el reúso de componentes.

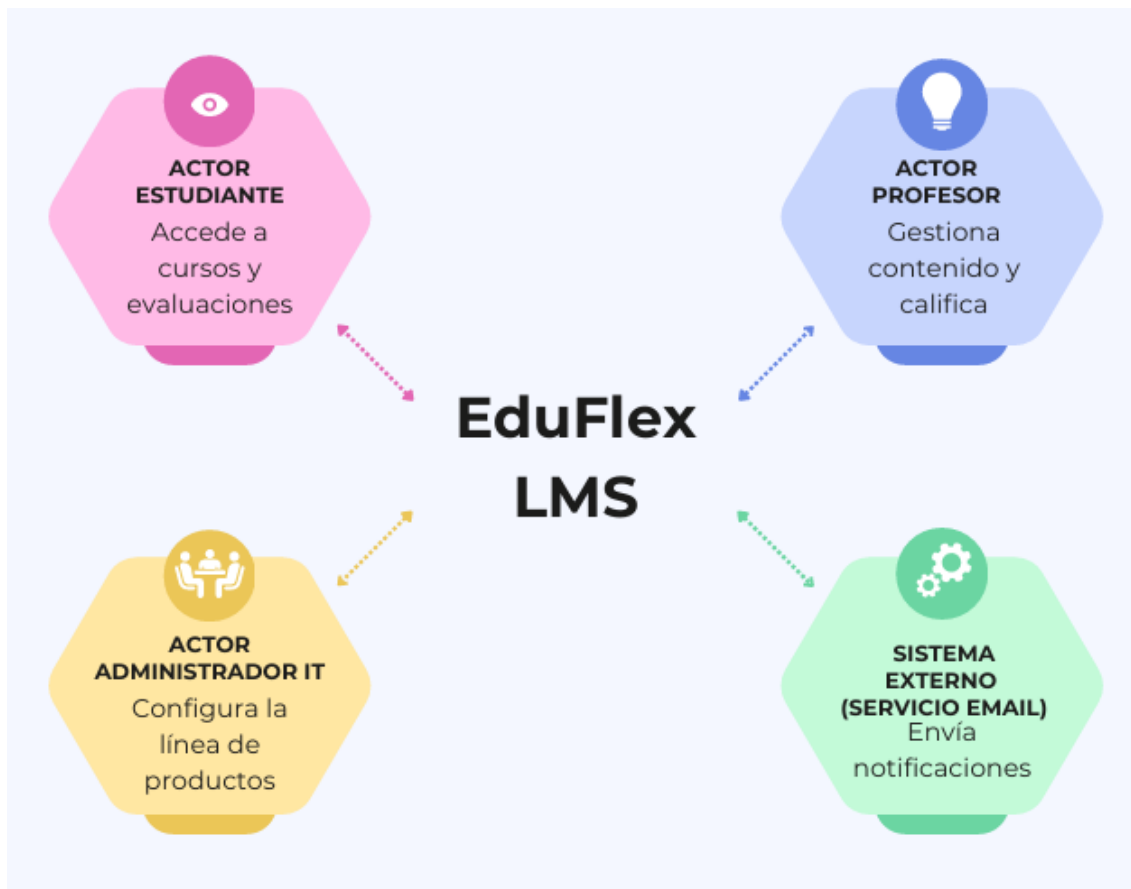


Fig. 2. Diagrama de Contexto (C4 Nivel 1). Fuente: Elaboración propia.

Diagrama C4 level 2 (Contenedores)

La arquitectura interna se compone de:

1. **Web Frontend:** Aplicación SPA (Single Page Application).
2. **API Gateway & Backend Core:** Desarrollado en ASP.NET Core 8.0, actuando como el motor de reglas de negocio.
3. **Base de Datos:** SQL Server en Microsoft Azure para alta disponibilidad.

El ecosistema de interacción y los componentes internos del sistema se describen en la **Fig. 3** (Contenedores) estableciendo una arquitectura modular compatible con el reúso de componentes.

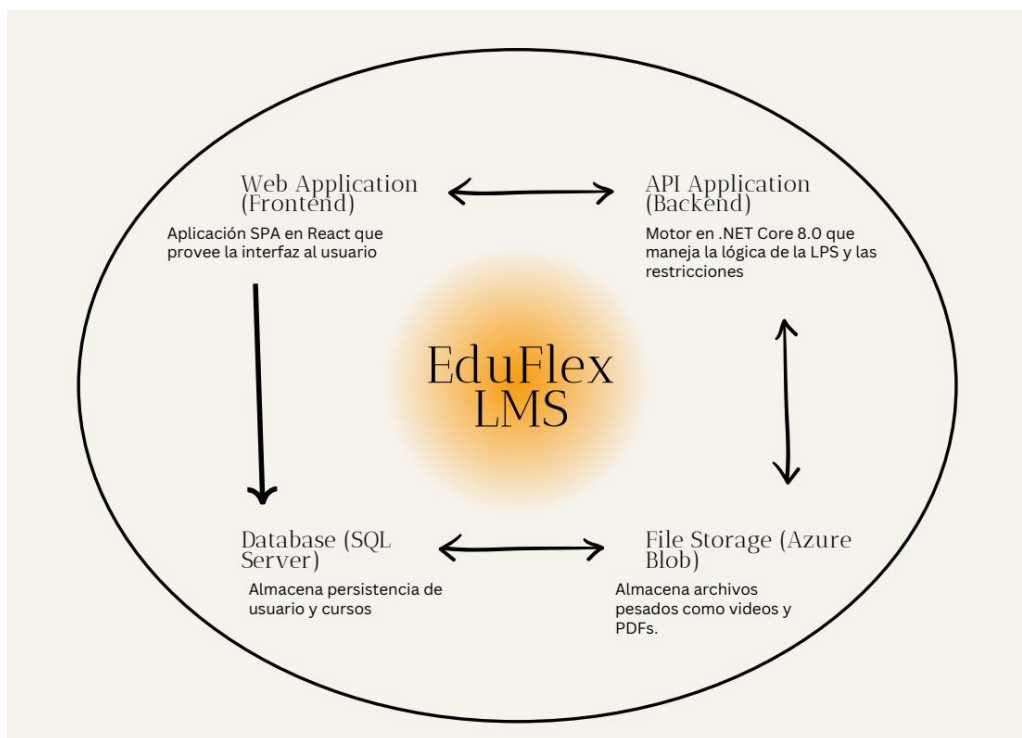


Fig. 3. Diagrama de Contenedores (C4 Nivel 2). Fuente: Elaboración propia.

Tabla de variation points

Los puntos de variación arquitectónica y sus mecanismos de resolución se detallan en la Tabla II.

TABLA II. PUNTOS DE VARIACIÓN

Variation Point (VP)	Variantes Posibles	Mecanismo de Variación	Binding Time
Infraestructura	On_Premise, SaaS	Configuración de despliegue (Docker/Azure)	Load-time
Pasarela de Pago	Stripe, Ninguna	Inyección de Dependencias (DI)	Compile-time
Comunicación	Foros, Video, Chat	Feature Toggles en el Backend	Run-time
Autenticación	RBAC local, SSO, LDAP	Patrón <i>Strategy</i> en ASP.NET Core	Run-time

Fuente: Elaboración propia.



5. Configuraciones de productos

Sección por producto

Producto 1: School Basic

- **Mercado:** Escuelas de educación primaria y secundaria (K-12).
- **Decisiones:** On_Premise = true, Forums = true, Gamification = true, Video_Conf = false, Payments = false.

Producto 2: University Pro

- **Mercado:** Instituciones de educación superior y posgrados.
- **Decisiones:** Cloud_SaaS = true, Video_Conf = true, Plagiarism_Check = true, Gamification = false.

Producto 3: Corporate Ent

- **Mercado:** Capacitación corporativa y B2B.
- **Decisiones:** Cloud_SaaS = true, Learning_Analytics = true, Payments = true, SSO_Login = true.

Tabla de decisiones

Las decisiones de selección de características para cada producto derivado se ilustran en la Tabla III.

TABLA III. MATRIZ DE DECISIONES POR PRODUCTO

Feature / Característica	School Basic	University Pro	Corporate Ent
Hosting: Cloud_SaaS		✓	✓
Hosting: On_Premise	✓		
Video_Conference		✓	
Plagiarism_Check		✓	
Learning_Analytics	✓		✓
Billing_Gateway			✓
SSO_Login			✓
Digital_Badges	✓		

Fuente: Elaboración propia.

Configuración válida - Captura de FeatureIDE

La validación de las tres variantes principales (School, University y Corporate) se confirma en la **Fig. 4**, evidenciando que la derivación de cada producto es consistente con las restricciones lógicas programadas.

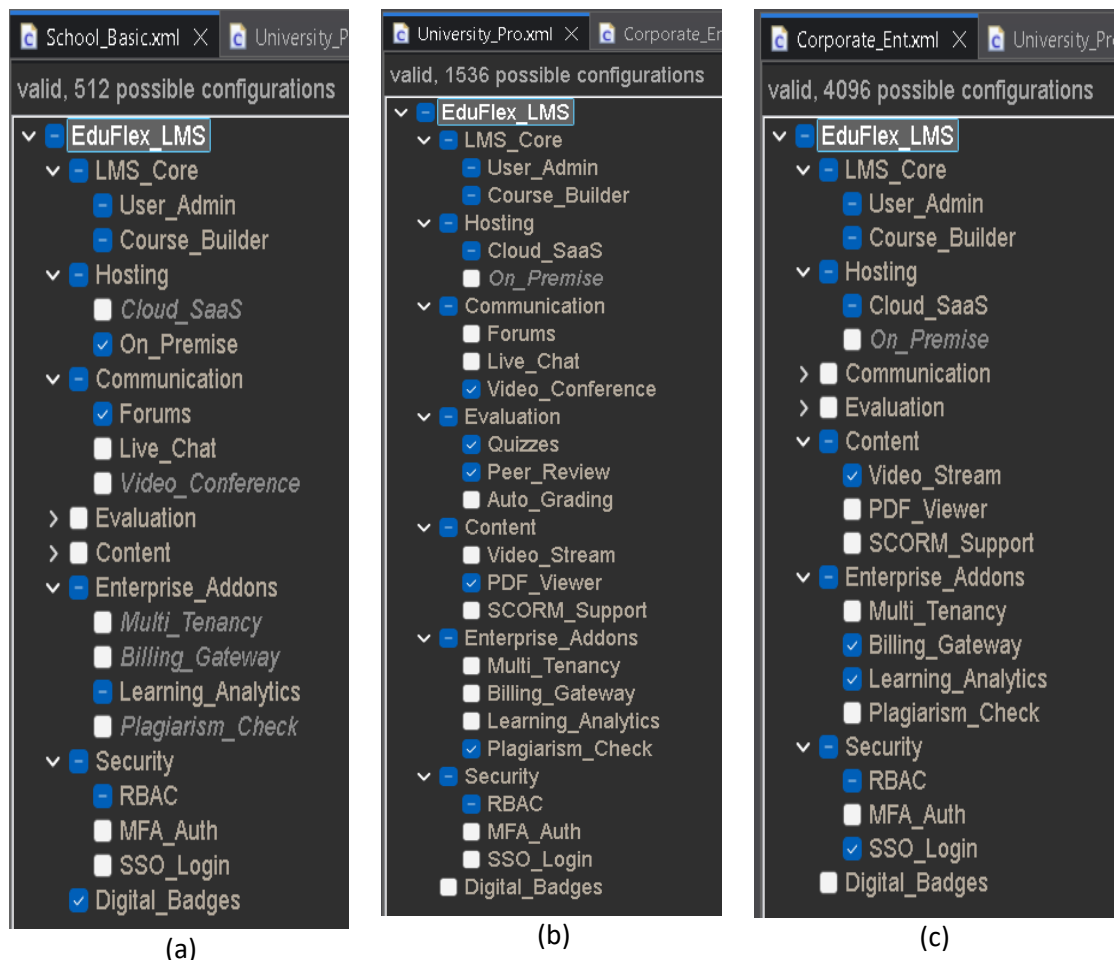


Fig. 4. Capturas de validación de configuraciones en FeatureIDE. Fuente: Elaboración propia.

6. Configuration Knowledge

El mapeo entre las características abstractas y los artefactos físicos de implementación (.cs, .sql, .json) se detalla en la **Tabla IV** que a continuación se muestra:

TABLA IV. MAPEADO DE CONFIGURATION KNOWLEDGE

Feature	Artefactos Incluidos/Excluidos	Artefactos Modificados
Video_Conf	(+) Integrations/ZoomService.cs	Modifica appsettings.json para activar credenciales de API.
Payments	(+) Billing/StripeController.cs	Modifica UserSchema.sql añadiendo historial de transacciones.
Gamification	(+) UI/BadgesComponent.jsx	Modifica menú de navegación en Sidebar.jsx.
Plagiarism_C	(+) Services/TurnitinAPI.cs	Modifica la lógica de la clase AssignmentSubmit.cs.
On_Premise	(-) CloudSetup/AzurePipeline.yaml	Habilita scripts de despliegue local en IIS.

Fuente: Elaboración propia.

7. Análisis de ROI y conclusiones

Modelo económico

La adopción de la LPS se justifica financieramente mediante el cálculo del *Break-even point*. El costo total de la línea de productos se formula como:

$$C_{LPSO} = C_{org} + \sum (C_{cab} + C_{reuse}) [2].$$

Dado que el costo de desarrollar el diseño base C_{org} es alto inicialmente, la inversión se recupera al compararla con el desarrollo *ad-hoc* de múltiples plataformas $n \times C_{ah}$.

Considerando los costos promedio del mercado, el punto de equilibrio se alcanza en $n = 3$ productos. A partir del cuarto cliente, el margen de beneficio por licenciamiento de software aumenta drásticamente

Para el cálculo del **Break-even point** (modelo de Clements & Northrop):

- C_{org} : Costo de crear la plataforma base o arquitectura de referencia (los "Core Assets").
- C_{cab} : Costo de desarrollar las partes únicas o características específicas que requiere un producto particular y que no están en la base.
- C_{reuse} : Costo de adaptar, integrar y probar los componentes que sí se están reutilizando de la plataforma base.
- C_{ah} : Costo de hacer una plataforma desde cero (ad-hoc).



UNIVERSIDAD TÉCNICA DEL NORTE

FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS

CARRERA DE SOFTWARE

- **Justificación:** Al alcanzar 3 productos vendidos, el costo de la línea de productos (C_{LPSO}) se iguala al desarrollo tradicional ($n \times C_{ah}$), y a partir del cuarto cliente, todo representa ganancia neta.

Lecciones aprendidas

Se evidenció que el análisis de dominio (FODA) es la etapa más crítica; un error en la definición de las reglas lógicas puede generar "features muertos" o configuraciones inválidas. Además, el reúso de componentes requiere una arquitectura modular estricta (como la inyección de dependencias) para evitar un alto acoplamiento de código.

¿Qué haría diferente?

Para futuras iteraciones, integraría herramientas de despliegue continuo (CI/CD) directamente vinculadas al modelo de características para automatizar la derivación del producto no solo a nivel de arquitectura, sino hasta el despliegue en la nube.

Nota de elaboración: Para la realización de este trabajo y la mejora de su redacción, se contó con el apoyo de herramientas de Inteligencia Artificial (IA).

IA utilizada: Gemini Google.

CONCLUSIONES

- **Eficiencia en el ciclo de vida:** El uso de FeatureIDE y la notación FODA demostró que centralizar la gestión de variabilidad reduce la complejidad técnica y evita la duplicación masiva de código común.
- **Robustez arquitectónica:** La implementación de *Cross-Tree Constraints* garantiza que los analistas funcionales no puedan derivar plataformas educativas técnicamente inviables o con dependencias rotas, asegurando la calidad del software.
- **Rentabilidad comprobada:** El análisis económico confirmó que, a pesar de requerir mayor tiempo y presupuesto en las fases iniciales de diseño, el modelo LPS es la única estrategia sostenible financieramente para escalar un producto tipo SaaS a múltiples segmentos de mercado educativo.

REFERENCIAS

- [1] K. Kang, S. Cohen, J. Hess, W. Novak, y A. Peterson, "Feature-Oriented Domain Analysis (FODA) Feasibility Study," Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, EE. UU., Inf. Téc. CMU/SEI-90-TR-21, 1990.
- [2] P. Clements y L. Northrop, *Software Product Lines: Practices and Patterns*. Boston, MA, EE. UU.: Addison-Wesley, 2001.
- [3] T. Thüm *et al.*, "FeatureIDE: An extensible framework for feature-oriented software development," *Science of Computer Programming*, vol. 79, pp. 70-85, ene. 2014.
- [4] S. Brown, *Software Architecture for Developers*. Leanpub, 2012.